

# SmartPRS Cloud (L3) — Multi-Tenant Remediation

Attendance ingest hardening · engineering and deployment record

| Field             | Detail                                                                                             |
|-------------------|----------------------------------------------------------------------------------------------------|
| Tree remediated   | C:\laragon\www\smartprs-l3 — the Ametecs cloud deployment                                          |
| Reference tree    | C:\laragon\www\smartprs2 — ahead of L3; all code copied from here verbatim                         |
| Why it matters    | MULTI-TENANT: several customers share this one server and one database                             |
| Date              | 18 August 2026                                                                                     |
| Scope             | FIX 1 verify · FIX 2 quarantine/replay · FIX 3 migration · FIX 4–6 runbooks · FIX 7 design options |
| Completed in code | FIX 1 (verified), FIX 2, FIX 3, FIX 7                                                              |
| Requires a human  | FIX 4 (deploy), FIX 5 (device→tenant assignment), FIX 6 (index check)                              |
| Files changed     | 2 — 1 modified (+154 lines), 1 added                                                               |
| PHP target        | 8.2 (L3 pins it; smartprs2 requires ^8.3 — all copied code is 8.2-safe)                            |
| Owner             | Ejaz Hussain                                                                                       |

## The single most important thing in this document

**FIX 1 changed behaviour deliberately: a device with no tenant now matches NOBODY.** On the cloud, where every customer has a real tenant, every auto-registered device stops delivering until it is assigned. Section 6 lists exactly what to run and record.

Do this **immediately after the deploy**, not later. Until then those devices' punches are held safely in quarantine — not lost, but not visible in attendance either.

## Contents

---

(In Word: right-click and choose "Update Field" to fill in page numbers.)

## 1. Status board

| Fix | What                                                        | Status           | Who finishes it                       |
|-----|-------------------------------------------------------------|------------------|---------------------------------------|
| 1   | Symmetric tenant filter — stop cross-customer contamination | VERIFIED PRESENT | Done (by another engineer, 03:04 IST) |
| 2   | Quarantine + replay — stop deleting unmapped punches        | DONE             | Done in this session                  |
| 3   | Missing migration (device_code column + index)              | DONE             | Done in this session                  |
| 4   | Deploy to the cloud and migrate                             | NOT DONE         | Human — runbook in §5                 |
| 5   | Assign a tenant to every auto-registered device             | NOT DONE         | Human — worksheet in §6               |
| 6   | Check two indexes on the cloud database                     | NOT DONE         | Human — §7, report only               |
| 7   | /iclock has no authentication — options write-up            | DELIVERED        | Decision needed — §8                  |

### What "not done" means here

FIX 4, 5 and 6 need a shell on the cloud server and access to the cloud database. This session has neither — it can only read and write files in the two local folders. Every syntax check and grep result quoted in this document was run in an isolated sandbox against a staged copy of the file, not on the cloud.

Sections 5–7 give the exact commands, the SQL, and a worksheet to record results. **Paste the output back and it can be interpreted** — particularly FIX 6, which has a trap.

## 2. What existed on L3 before this session

### 2.1 The gap against the reference tree

L3 had already received FIX 1 but not the rest. Measured, not assumed:

| Item                                | L3 before                | smartprs2 reference      |
|-------------------------------------|--------------------------|--------------------------|
| app/Services/ETimeOfficeService.php | 471 lines / 19,699 bytes | 625 lines / 26,072 bytes |
| Symmetric tenant filter (FIX 1)     | PRESENT                  | present                  |
| quarantine()                        | ABSENT                   | present                  |
| promotePending()                    | ABSENT                   | present                  |
| Call site inside import()           | ABSENT — bare continue   | present                  |
| recordUnmapped()                    | present, byte-identical  | present                  |
| 2026_08_18_000100 migration         | ABSENT                   | present                  |
| All 45 other migrations             | present                  | present                  |

### 2.2 What that meant in practice

#### Every unmapped punch on L3 was being destroyed

Without `quarantine()`, a punch arriving on `/iclock` whose PIN matched no employee hit a bare `continue`. The punch was gone permanently. Re-sending could not recover it, because the loss is on our side, not the sender's.

The HTTP response was still **200**, so nothing reported the loss. Only a counter survived, via `recordUnmapped()`. The customer would discover it at payroll.

This bites hardest at go-live — the exact moment when nobody is mapped yet.

## 3. What was fixed

### 3.1 FIX 1 — verified only, not modified

The three prescribed checks, run against L3's file:

```
php -l app/Services/ETimeOfficeService.php
-> No syntax errors detected

grep -c "if ($tid && Schema::hasColumn('employees', 'tenant_id'))"
-> 0          (expected 0)    PASS

grep -c "$tid ? $q->where('tenant_id', $tid) : $q->whereNull('tenant_id');"
-> 2          (expected >=2)  PASS
```

Applied by another engineer at **03:04 IST on 18 August**. The tenant filter was not touched in this session.

#### A caveat worth recording about that second check

`grep -c` on the symmetric pattern returns **2 even on a completely unpatched file**, because that idiom already existed in `configForTenant()` and `recordUnmapped()`. It is not evidence of the fix on its own. Only the first `grep` — the count of the OLD conditional — actually proves it. That count is 0, so FIX 1 is genuinely in place.

### 3.2 FIX 2 — quarantine and replay

One file changed: `app/Services/ETimeOfficeService.php`. **+154 lines, 0 removed**. 471 → 625 lines. `php -l` clean.

| Added                                                | At line | Purpose                                                                           |
|------------------------------------------------------|---------|-----------------------------------------------------------------------------------|
| <code>use Illuminate\Support\Facades\Log;</code>     | 9       | Required — <code>promotePending()</code> logs promotions                          |
| <code>private static function quarantine()</code>    | 409     | Holds an unmatched punch in <code>attendance_pending</code>                       |
| <code>public static function promotePending()</code> | 454     | Replays held punches when a code is mapped                                        |
| <code>self::quarantine(\$p, \$cfg, \$full);</code>   | 573     | The call, inside <code>import()</code> , where the bare <code>continue</code> was |
| <code>recordUnmapped()</code>                        | 354     | Compared — byte-identical, nothing to bring across                                |

#### How the copy was verified

The functions were copied at line level using the exact diff hunks between the two files — not retyped, not adapted. The result was then diffed **back** against the reference. The only remaining differences are **three comment lines**, where L3's FIX 1 comments use ASCII hyphens and the reference uses em-dashes. Those sit inside the FIX 1 block that must not be re-touched, so they were left exactly as they are. Everything else is byte-for-byte identical to `smartprs2`.

**No wholesale file copy was made.** The rest of L3's file — including its own FIX 1 wording — is untouched.

### 3.3 FIX 3 — the missing migration

`database/migrations/2026_08_18_000100_attendance_pending_device_code.php` (5,381 bytes) copied unchanged. Byte size verified against the reference. Additive and safe to run twice.

**Directory diff — exactly one file was missing, nothing else:**

```
diff <(ls ../smartprs2/database/migrations) <(ls database/migrations)

< 2026_08_18_000100_attendance_pending_device_code.php

(all 45 other migrations match by name in both trees)
```

**A correction, and a lesson about L3**

This same migration was written to L3 on the night of 17 August and the write reported success. **It was not present this morning.** L3 was evidently re-synced or redeployed afterwards — consistent with FIX 1 arriving from another hand at 03:04.

**Do not trust a prior "file delivered" report for L3, including one from this session.** Verify the tree after every deploy or package rebuild.

## 4. Impact

### 4.1 Before and after, on this server

| Scenario                                                          | Before                                   | After                                                   |
|-------------------------------------------------------------------|------------------------------------------|---------------------------------------------------------|
| Device assigned to tenant 7, PIN maps to a tenant 7 employee      | Filed correctly                          | Filed correctly — unchanged                             |
| Device assigned to tenant 7, PIN maps to nobody                   | <b>Punch destroyed</b>                   | Held in attendance_pending, replayed on mapping         |
| Device with NO tenant, PIN matches some other customer's employee | <b>Filed against the WRONG customer</b>  | Matches nobody → held safely                            |
| Admin maps a code to an employee                                  | Nothing to replay — punches already gone | Held punches promoted, backdated to the real punch time |
| Same device re-sends a batch                                      | Duplicate risk                           | Quarantine is idempotent; promotion uses updateOrInsert |

### 4.2 The deliberate side effect you must act on

#### Unassigned devices now deliver nothing until assigned

This is the correct trade — **a held punch beats a punch on the wrong customer's payroll** — but it is not self-healing. Section 6 is not optional.

Symptom if it is skipped: a customer reports "attendance stopped" while the server logs look healthy and return 200. The punches will be sitting in attendance\_pending, recoverable in full once the device is assigned and the codes mapped.

### 4.3 What was NOT changed

- PushController and the /iclock routes — real ZKTeco hardware and SBB both depend on them.
- The auto-register behaviour — devices still appear on first contact.
- The FIX 1 tenant filter — verified only.
- Any row in attendance\_logs. No payroll data was read or written by this session.
- Every other difference between L3 and smartprs2 — left alone by instruction.

## 5. FIX 4 — deploy and migrate (human)

### Order matters — migrate before any device sends

`promotePending()` and the Pending Punches panel both read `attendance_pending.device_code`, and that column only exists after the FIX 3 migration runs. Code without migration means quarantine writes fail — fail-soft and logged, so punches would be silently lost again, which is the very thing being fixed.

```
# 0. BACK UP FIRST. This is a multi-tenant database.
mysqldump -u <user> -p <dbname> > l3_backup_2026-08-18.sql

php artisan down

# deploy the code (git pull / patch zip / rsync – whatever your process is)
composer install --no-dev --optimize-autoloader

php artisan migrate --force

php artisan config:clear && php artisan route:clear && php artisan cache:clear
php artisan up
```

### 5.1 Confirm the deploy actually landed

Because L3 has lost a delivered file before (§3.3), verify rather than assume:

```
grep -c "function quarantine"      app/Services/ETimeOfficeService.php # expect 1
grep -c "function promotePending"  app/Services/ETimeOfficeService.php # expect 1
grep -c "self::quarantine("        app/Services/ETimeOfficeService.php # expect 1
ls database/migrations/ | grep 2026_08_18 # expect 1
file
php artisan migrate:status | grep 2026_08_18 # expect
"Ran"
```

## 6. FIX 5 — assign a tenant to every auto-registered device (human)

Run immediately after FIX 4. Every row this returns is a device that will now match nobody.

```
SELECT id, serial_number, label, tenant_id, last_sync_at, last_status
FROM biometric_configs
WHERE tenant_id IS NULL
ORDER BY last_sync_at DESC;
```

### 6.1 Worksheet — one row per device

Identify each serial against the customer record. **Do not guess.** If a device cannot be identified, leave it and report it — an unassigned device holds its punches safely; a wrongly assigned one files them into another customer's payroll.

| id | serial_number | Identified customer | tenant_id | company_id | Evidence used | Assigned by / time |
|----|---------------|---------------------|-----------|------------|---------------|--------------------|
|    |               |                     |           |            |               |                    |
|    |               |                     |           |            |               |                    |
|    |               |                     |           |            |               |                    |
|    |               |                     |           |            |               |                    |
|    |               |                     |           |            |               |                    |
|    |               |                     |           |            |               |                    |

### 6.2 Two ways to assign

#### Preferred — through the UI

Settings → **Pending Punches** → "Unassigned devices" card → choose the company → **Assign**. This route is guarded: only a row whose `tenant_id` is NULL can be claimed, so one tenant can never take another's device. It also logs who did it.

#### If the UI is unavailable — SQL, one device at a time

```
-- Confirm you have the RIGHT row first.
SELECT id, serial_number, label FROM biometric_configs WHERE id = <id>;

UPDATE biometric_configs
SET tenant_id = <tenant_id>, company_id = <company_id>, updated_at = NOW()
WHERE id = <id> AND tenant_id IS NULL;      -- the NULL guard is deliberate
```

Never run a blanket UPDATE ... WHERE `tenant_id IS NULL` across all rows. One statement, one device, verified each time.

### 6.3 After assigning — release the held punches

1. Open Settings → Pending Punches. Held punches are grouped by device code and device serial.
2. Map each code to the right employee. The Map action passes the serial, so two devices that both use PIN 5 stay separate.

3. Held punches are promoted into attendance immediately, backdated to when they actually happened.
4. Confirm in Attendance that the backdated punches appear at their original times.

## 7. FIX 6 — check two indexes (report only, do not create)

```
SHOW INDEX FROM attendance_logs
WHERE Key_name IN ('attlog_tenant_external_unique', 'attlog_natural_unique');
```

### 7.1 How to read the result

| Result                         | Meaning                                                                  | Action                                                       |
|--------------------------------|--------------------------------------------------------------------------|--------------------------------------------------------------|
| Both index names present       | The repair migration succeeded fully                                     | Record it. This is what unblocks SBB retries.                |
| Either name missing            | CREATE UNIQUE INDEX failed on duplicate rows and the error was swallowed | <b>REPORT ONLY. Do not create it during a deploy window.</b> |
| Table has no such index at all | Same as above                                                            | Same                                                         |

### 7.2 Why a missing index can look like a successful migration

2026\_08\_17\_000100\_repair\_sbb\_schema.php wraps index creation in a catch:

```
} catch (\Throwable $e) {
    // Never abort the repair on an index...
    report($e);
}
```

#### A caught exception is not a success

If attendance\_logs already holds duplicate rows, the CREATE UNIQUE INDEX fails, the exception is caught and logged, and **the migration still reports success**. migrate:status will say "Ran". Only SHOW INDEX tells the truth.

This pattern was written deliberately — so a deploy is never blocked by an index — but it means the index must be confirmed separately, every time.

### 7.3 Why it matters commercially

Smart Biometric Bridge re-sends any batch it did not see acknowledged. Without attlog\_tenant\_external\_unique, a re-send duplicates every punch that already landed. **SBB currently refuses to retry into SmartPRS for exactly this reason**. Confirming the index exists is what allows that restriction to be lifted — which in turn is what makes the bridge resilient to a dropped connection.

Creating the index is a separate, planned piece of work: it requires deduplicating live payroll data first, and a failed CREATE UNIQUE INDEX part-way through cannot be rolled back cleanly. There is also a known mismatch to resolve first — the existing cleanup compares keys **case-sensitively in PHP** while MySQL compares **case-insensitively**.

## 8. FIX 7 — /iclock has no authentication: options

Nothing was changed. This section is a decision paper.

### 8.1 The exposure

routes/web.php:92 registers /iclock/cdata with throttle:1000,1 and nothing else — no authentication, no CSRF, no IP restriction. Anyone who can reach the server can post attendance punches into it.

FIX 1 materially reduced the damage: an unauthenticated poster using an unknown serial now fails closed into quarantine rather than landing in an arbitrary customer's payroll. **But two risks remain:**

- **Quarantine flooding** — an attacker can fill attendance\_pending and the admin UI with noise.
- **Serial guessing** — a serial number is printed on the device and travels in the URL and server logs. It is an identifier, not a secret. Anyone who learns a \*registered\* serial can post punches that WILL be filed against that customer.

#### The constraint that shapes every option

Real ZKTeco firmware lets you set **host and port only**. It cannot add a header, cannot present a certificate, and in most models cannot change the URL path. So no in-app credential scheme can cover real hardware. **SBB is different — it is our own software and can authenticate however we like.** That asymmetry is the key to the recommendation.

### 8.2 The options

| Option                        | How it works                                                                                         | Pros                                                                                                     | Cons / cost                                                                                                                  |
|-------------------------------|------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| A. Serial allow-list          | Reject any SN not pre-registered by an admin. Turn auto-register into "record as pending, disabled". | No device config. Stops random posters immediately. Small change. Composes with work already done.       | SN is not a secret — a leaked SN still gets in. Loses plug-and-play; every install needs the SN entered first.               |
| B. Per-device shared secret   | A token the device sends with each upload.                                                           | A real credential, per device, revocable.                                                                | <b>Not possible on real ZKTeco hardware</b> — the firmware cannot send one. Viable for SBB only.                             |
| C. IP restriction at the edge | Firewall / reverse proxy allows /iclock only from each customer's known egress IP.                   | Strongest control available for dumb hardware. No app change. Satisfies a security review cleanly.       | Needs a static IP per site. Breaks on 4G/dynamic links. Per-site ops burden; a customer IP change silently stops attendance. |
| D. mTLS at the proxy          | Client certificate required.                                                                         | Very strong. Standard for machine-to-machine.                                                            | Hardware cannot do it. SBB could. Certificate lifecycle to manage.                                                           |
| E. Move traffic to /api/v1    | Point SBB at the authenticated JSON API already built and tested.                                    | Real scoped keys, per-punch verdicts, database-level idempotency, expiry and revocation. Already exists. | Only helps where SBB is the sender. Real hardware still needs A or C.                                                        |

### 8.3 Recommendation — layered, in this order

1. **Move every SBB site onto `/api/v1`** (option E). It is built, tested, and gives real authentication today. This shrinks `/iclock` to hardware-only traffic.
2. **IP-restrict `/iclock` at the edge** (option C) for sites with a static IP. No code, strongest control for dumb devices.
3. **Add a serial allow-list** (option A) as defence in depth, and change auto-register so an unknown serial is recorded **disabled** — visible in the Unassigned devices card, but unable to inject until an admin enables it. This closes the flooding risk and keeps the plug-and-play discovery that makes installs easy.
4. Keep the throttle, and alert on a spike in quarantine volume per serial — that is what an attack looks like now that FIX 1 makes it fail closed.

Item 3 is the smallest change with the largest security return, and it composes with FIX 1 and FIX 2 rather than replacing them. **It should not be attempted before the deliveries** — it changes first-contact behaviour for every device.

## 9. Testing

### 9.1 What was verified in this session

| Check                  | Method                                          | Result                                          |
|------------------------|-------------------------------------------------|-------------------------------------------------|
| FIX 1 present on L3    | The three prescribed commands, on a staged copy | PASS — 0 / 2 / no syntax errors                 |
| New file parses        | php -l on the rebuilt ETimeOfficeService.php    | PASS                                            |
| Copy fidelity          | Diff of the result back against smartprs2       | PASS — only 3 pre-existing comment lines differ |
| recordUnmapped parity  | Function extracted from both files and diffed   | PASS — byte-identical                           |
| Migration fidelity     | Byte-size comparison against the reference      | PASS — 5,381 bytes both sides                   |
| Migration completeness | Directory listing diff, both trees              | PASS — 1 file was missing, now copied           |

#### The limit of the above

All of it is static verification run in an isolated sandbox. **No test suite was executed against L3, and no code was run on the cloud.** The behavioural proof lives in tests/Feature/IclockTenantHotfixTest.php in the reference tree — 11 cases covering tenant isolation, the single-tenant regression, quarantine, device-scoped replay and re-import safety. Run it on the development box before deploying L3.

### 9.2 Run the behavioural tests (development box)

```
cd C:\laragon\www\smartprs2
RUN-SBB-TESTS.bat
vendor\bin\pest tests/Feature/IclockTenantHotfixTest.php
```

The case that matters most is **test 3** — it proves a single-tenant install behaves exactly as before, which is the regression protecting the self-hosted customers. Tests 1 and 2 are the ones that protect this cloud server.

### 9.3 Post-deploy smoke test on the cloud

| # | Step                                                 | Expected                                              | Pass |
|---|------------------------------------------------------|-------------------------------------------------------|------|
| 1 | GET /iclock/cdata?SN=TESTSN&options=all              | Option block returned — hardware path alive           |      |
| 2 | Post a punch from a real assigned device, PIN mapped | Appears in Attendance for the right customer          |      |
| 3 | Post a punch with an unmapped PIN                    | Appears in Pending Punches, NOT lost                  |      |
| 4 | Map that code to an employee                         | Held punch appears in Attendance at its original time |      |
| 5 | Re-send the same punch                               | No duplicate row                                      |      |
| 6 | Check an unassigned device                           | Its punches go to quarantine, none reach any customer |      |

| # | Step                                | Expected                                      | Pass |
|---|-------------------------------------|-----------------------------------------------|------|
| 7 | Confirm no other tenant gained rows | Row counts unchanged for every other customer |      |

## 10. Risks and sign-off

### 10.1 Open risks

| #  | Risk                                                     | Severity | Handling                                                                                                    |
|----|----------------------------------------------------------|----------|-------------------------------------------------------------------------------------------------------------|
| R1 | Unassigned devices stop delivering after FIX 1           | HIGH     | Deliberate. \$6 must be completed immediately after deploy. Punches are held, not lost.                     |
| R2 | attendance_logs unique indexes may be silently absent    | HIGH     | \$7. Report only — creating them requires deduplicating live payroll data first.                            |
| R3 | /iclock remains unauthenticated                          | MEDIUM   | \$8. Design decision required; nothing changed.                                                             |
| R4 | L3 has lost a delivered file before                      | MEDIUM   | Verify the tree after every deploy — \$5.1.                                                                 |
| R5 | PushController::parseDate keeps a sender timezone offset | MEDIUM   | Unchanged. Confirm SBB sends Y-m-d H:i:s with no offset. The /api/v1 path already rejects offsets outright. |
| R6 | smartprs-l1 and smartprs-l2 are not in sync              | LOW      | Sync before either builds a client package.                                                                 |

### 10.2 Deployment gate

| # | Gate                                                          | Owner      | Done |
|---|---------------------------------------------------------------|------------|------|
| 1 | IclockTenantHotfixTest green on the development box           | Dev        |      |
| 2 | Cloud database backed up                                      | Deployment |      |
| 3 | FIX 4 deploy + migrate completed                              | Deployment |      |
| 4 | Post-deploy file verification (\$5.1) passed                  | Deployment |      |
| 5 | FIX 5 worksheet completed — every device assigned or reported | Deployment |      |
| 6 | FIX 6 index result recorded and reported                      | Dev        |      |
| 7 | Smoke test \$9.3 passed                                       | Deployment |      |
| 8 | FIX 7 option chosen and scheduled                             | Ejaz       |      |

### 10.3 Approval

| Role                   | Name         | Signature | Date |
|------------------------|--------------|-----------|------|
| Engineer               |              |           |      |
| Deployment lead        |              |           |      |
| Approved for the cloud | Ejaz Hussain |           |      |